The `find` method is not declared *inside* the class. The class body has a function scope, so the `find` method must be defined inside it in order to see the `product` variable. How do you define it *inside* the class body so that all the things related to the class are defined in one place?

5.3.2. Declaring (to keep things together)

Inside a constructor, @ refers to the new object being constructed. Inside an instance method, @ refers to the instance by default. That much you know. Directly inside the class declaration, @ refers to the class itself. The @ keyword is used to define the `find` property on the `Product` class:

```
class Product
  @find = (what) ->
    "#{what} not found"

Product.find "zombie survival kit"
# "zombie survival kit not found"
```

Directly inside the class declaration, @ refers to the class.

Remember that @ is dynamic; the object it refers to depends on how the function is invoked. Inside the class body the class body itself has been invoked. Yes, the class body works like a function. It's still too soon to see why, and besides you now have a solution for `Product.find`:

```
class Product
  instances = []
  @find = (query) ->
    (product for product in instances when product.name is query)
  constructor: (name) ->
    instances = instances.concat [@]
    @name = name
```

Class variable

Class method

Adding the new instance to the array of instances using the array concat method (section 2.6.1)

Now, by using a class variable and a class method, you have a solution that finds a product:

```
new Product "Green", {}
Product.find 'Green'
# [ { name: 'Green' } ]
```